Voucher Management System

RP2

PROJET VMS

Par
Yeshnav Sookur

Voucher Management System

Secure access for finance, approvals, and redemption operations.

Fait à: MCCI Business School
Période: 2025–2026

Sommaire

01 Introduction

Presentation de l'organisation cliente 04

Besoins Éxprimés 05

Enjeux 06

Missions 07

Organisation du travail 08

02 Conception et planification

Analyse des besoins 10

Étapes de conception 12

Conception orientée règles de gestion 14

03 Réalisation technique

Architecture globale 15

Base de données 16

Difficultés rencontrées et solutions 17

04 Résultats obtenus

Présentation de l'application finale 18

Conformité aux besoins exprimés 19

Évaluation qualitative 20

Conclusion

Annexe 21

Introduction

La digitalisation des processus de distribution de bons d'achat est devenue une priorité pour les organisations souhaitant améliorer la traçabilité, réduire les erreurs manuelles et accélérer les opérations internes. Dans de nombreux environnements, la gestion des vouchers se fait encore de manière semi-manuelle, avec un risque élevé de doublons, de perte d'information et de manque de visibilité sur le cycle de vie des demandes.

Le projet VMS (Voucher Management System) répond à cette problématique en proposant une application métier centralisée couvrant l'ensemble du flux : création de demandes, validation des paiements, approbation, génération des vouchers au format PDF, envoi automatisé au client et utilisation en point de vente via un QR code.

Ce rapport présente de façon détaillée le contexte, la conception, la réalisation technique ainsi que les résultats obtenus pour ce projet. Il met également en avant les choix technologiques effectués, les difficultés rencontrées, les solutions apportées et les perspectives d'amélioration.

Partie 1- Contexte du projet

1.1 Presentation de l'organisation cliente (VMS)

L'organisation cliente exploite un modèle de distribution de vouchers destiné aux partenaires, aux clients et aux points de vente. Dans ce contexte, la valeur du voucher n'est pas uniquement commerciale : elle implique également des exigences en matière de sécurité, de validation comptable et de contrôle d'usage.

Le périmètre métier du VMS couvre principalement :

- La création et la gestion des demandes de vouchers
- La vérification de la preuve de paiement
- L'approbation par un profil habilité
- La génération et la diffusion des vouchers
- Le suivi de l'état des vouchers (actif, redeem, expiré)
- L'exploitation de rapports de suivi pour le pilotage.

Activité de l'entreprise

L'activité se situe à l'interface entre l'administration commerciale et l'exécution opérationnelle. Plus concrètement :

- Les comptables (Accountants) traitent les demandes et les paiements
- Les approbateurs (Approvers) valident la conformité avant émission
- Les superviseurs (Supervisors) effectuent la redemption en magasin
- Les super-utilisateurs (Superusers) administrent les données de référence (clients, branches, utilisateurs) et supervisent l'ensemble du système.

Le système doit ainsi garantir un cloisonnement clair des responsabilités afin d'éviter les conflits de rôles et de renforcer la gouvernance.

Partie 1

Besoins exprimés

D'après les spécifications du projet et le workflow de référence, les besoins principaux étaient les suivants :

1. Centraliser les informations dans une base unique afin d'éviter la dispersion (emails, fichiers, tableurs)
2. Structurer un workflow de validation avec des étapes obligatoires (preuve de paiement → validation du paiement → approbation → génération)
3. Sécuriser les vouchers via un identifiant unique et un QR code vérifiable
4. Automatiser la génération documentaire avec la création de PDF standards
5. Automatiser la communication client via l'envoi des vouchers par email
6. Permettre une redemption contrôlée afin de limiter le double usage
7. Produire des indicateurs (reporting, export Excel) pour le suivi opérationnel.

Ces besoins montrent que le projet dépasse une simple interface CRUD. Il s'agit d'un système métier intégrant des règles de gestion explicites, des contraintes de rôles ainsi que des exigences élevées en matière de traçabilité.

Partie 1

1.1.1 Enjeux métier et risques en l'absence de système intégré

En l'absence d'un outil centralisé, le processus de gestion des vouchers expose l'organisation à plusieurs risques concrets :

• Risque financier : émission de vouchers sans validation complète du paiement ;
• Risque opérationnel : difficultés de suivi des statuts en temps réel ;
• Risque de fraude : réutilisation d'un voucher ou duplication non détectée ;
• Risque de satisfaction client : retards de traitement et erreurs documentaires ;
• Risque de gouvernance : exécution d'actions critiques sans séparation claire des responsabilités.

Le projet VMS doit ainsi être compris comme un outil de contrôle interne numérique, au service à la fois de la productivité et de la conformité.

Partie 1

1.2 Mission du projet

La mission consiste à concevoir et implémenter une application de gestion de vouchers de bout en bout, en respectant les contraintes suivantes :

• Rapidité d'utilisation pour les profils non techniques ;
• Respect strict du workflow de validation ;
• Intégrité des données et auditabilité ;
• Capacité d'évolution vers une phase 2 (redemption légère en point de vente).

Le projet vise également à standardiser la qualité du service rendu au client final : réduction des délais de traitement, production de documents homogènes et automatisation de la communication par email.

Rôle dans le projet

J'ai occupé un rôle de développeur full-stack applicatif orienté métier, avec des responsabilités transversales :

• J'ai réalisé l'analyse fonctionnelle des besoins
• J'ai modélisé les données
• J'ai développé l'interface desktop avec JavaFX
• J'ai intégré la base de données PostgreSQL
• J'ai intégré des bibliothèques tierces (ZXing, PDFBox, Jakarta Mail)
• J'ai effectué les tests fonctionnels des parcours utilisateurs
• J'ai produit des artefacts de démonstration (présentation, prototype).

Partie 1

1.3 Organisation du travail

L'organisation du travail a suivi une logique itérative et incrémentale. La progression s'est articulée autour de blocs fonctionnels (requests, validation, vouchers, redemption, reporting) afin d'obtenir des livrables rapidement exploitables.

Approche Kanban

L'approche Kanban a été retenue pour ce projet car elle facilite :

- La visualisation des tâches en cours
- La limitation du travail simultané
- Le suivi des blocages techniques
- La priorisation en fonction de la valeur métier.

Une structure Kanban type a été mise en place pour organiser le projet comme suit :

- Backlog : ensemble des besoins à traiter (fonctionnalités, corrections, qualité)
- To Do : tâches planifiées pour l'itération
- In Progress : travaux de développement en cours
- Review/Test : vérifications fonctionnelles et techniques
- Done : fonctionnalités validées.

Partie 1

1.3.1 Chronologie de réalisation (vision synthèse)

Une chronologie type du projet peut être présentée en six macro-phases :

1. Cadrage et collecte des besoins : définition du workflow cible et des profils
2. Mise en place du socle technique : structuration du projet Maven, configuration de la connexion à la base de données et mise en place des modèles
3. Développement du cœur métier : gestion des requests, des validations et des rôles
4. Ajout des intégrations : QR code, génération de PDF et envoi d'emails
5. Ajout des modules étendus : redemption via caméra, reporting et export Excel
6. Stabilisation et démonstration : corrections, présentation et documentation.

Ce schéma met en évidence une progression du cœur métier vers la valeur complète, plutôt qu'un simple empilement de modules isolés.

Partie 2- Conception et planification

2.1 Analyse des besoins et choix techniques

L'analyse des besoins a conduit à distinguer :

- **Besoins fonctionnels** : gestion des demandes, validation, génération des vouchers, redemption et reporting
- **Besoins non fonctionnels** : fiabilité, lisibilité des rôles, maintenabilité, simplicité d'usage et extensibilité.

À partir de cette analyse, les choix techniques suivants ont été retenus.

a) **Java 17 + JavaFX 21**

- Java 17 garantit la stabilité, la compatibilité avec les environnements d'entreprise et de bonnes performances
- JavaFX 21 permet de concevoir une interface desktop riche, réactive et moderne
- L'interface est construite sans FXML (centralisée dans MainApp.java), ce qui simplifie certains aspects du contrôle logique, bien que cela augmente la taille du fichier UI.

b) **PostgreSQL**

- SGBD robuste et adapté aux relations métier du projet
- Support des contraintes référentielles (clés étrangères)
- Bonne compatibilité avec JDBC
- Simplicité de déploiement en local ou à distance.

Partie 2

c) **Maven**

- Gestion unifiée des dépendances
- Build reproductible
- Lancement de JavaFX via un plugin dédié
- Packaging en JAR avec maven-shade-plugin.

d) **Bibliothèques métier**

- ZXing : génération et lecture de QR codes
- PDFBox : génération des vouchers au format PDF
- Jakarta Mail : envoi d'emails via SMTP
- Webcam Capture : gestion du flux caméra pour la redemption
- Apache POI : export des données de reporting vers Excel.

Le choix de cet écosystème met en évidence une priorité donnée à la maîtrise technologique, à un coût maîtrisé (solutions open source) et à une intégration rapide.

2.1.1 Justification des compromis technique

Tout projet appliqué nécessite des compromis. Les principaux arbitrages réalisés ont été les suivants :

- **Desktop JavaFX vs application web** : JavaFX offre un cycle de livraison plus direct dans ce contexte, sans imposer la mise en place d'une infrastructure web complète.
- **JDBC direct vs ORM** : JDBC est plus simple et plus transparent pour ce volume fonctionnel. Un ORM aurait apporté une abstraction supplémentaire, mais également une complexité accrue en termes d'apprentissage et de mapping.

Partie 2

UI centralisee vs architecture MVC stricte : le choix d'un fichier principal unique accelere la vitesse de developpement initiale au prix d'une dette de modularisation a moyen terme. Ces compromis sont coherents avec un objectif de livrable fonctionnel rapide et stable.

2.2 Étapes de conception

La conception s'est déroulée en plusieurs étapes successives afin de réduire les risques.

Étape 1 – Définition du workflow métier

Le flux cible a été formalisé de manière stricte :

1. Création de la demande ;
2. Upload de la preuve de paiement ;
3. Validation du paiement ;
4. Approbation ;
5. Génération des vouchers, PDF et envoi email ;
6. Redemption en point de vente.

Cette séquence a été volontairement contrainte afin d'éviter toute émission sans contrôle.

Étape 2 – Modélisation des données

Le schéma relationnel a été structuré autour des entités suivantes :

- users
- clients
- company
- branch
- request
- vouchers

Les relations (notamment requests → clients et vouchers → requests) assurent la cohérence métier et la traçabilité des opérations.

Partie 2

Étape 3 – Conception des rôles et permissions

Une matrice de permissions a été définie afin d'isoler les actions sensibles :

- Accountant : création, upload de preuve et validation du paiement ;
- Approver : approbation et génération des vouchers ;
- Superuser : administration globale du système ;
- Supervisor : redemption sur le terrain.

Cette séparation renforce la robustesse du système face aux erreurs de manipulation.

Étape 4 – Prototypage UI et parcours utilisateur

La navigation principale a été conçue sous forme de pages métier :

- Dashboard
- Requests
- Clients
- Vouchers
- Branches
- Users
- Reports
- Redeem

Le choix d'une navigation latérale facilite l'accès direct aux modules selon le rôle de l'utilisateur.

Étape 5 – Intégration des services transverses

Une fois le cœur CRUD stabilisé, les services avancés ont été intégrés :

- génération de QR codes ;
- génération de PDF ;
- notifications par email ;
- export Excel ;
- scan via caméra.

Cette approche incrémentale limite les régressions et permet une validation progressive bloc par bloc.

Partie 2

2.2.1 Conception orientée règles de gestion

Une caractéristique forte du projet est l'intégration des règles de gestion directement dans les interactions utilisateur. Chaque action sensible est soumise à des préconditions :

• impossibilité de marquer une demande comme payée sans preuve de paiement ;

• impossibilité d'approuver sans validation du paiement ;

• impossibilité de générer les vouchers avant approbation ;

• impossibilité de retraiter certains statuts une fois finalisés.

Cette logique permet de réduire les erreurs de manipulation et de transférer les contraintes métier au niveau du système, ce qui est préférable à un simple contrôle procédural humain.

2.3 Outils utilisés

IntelliJ IDEA est utilisé comme IDE principal pour le développement Java, le debug, la gestion Maven et l'exécution de l'application.

GitHub assure le versioning, le suivi des évolutions et la gestion du projet.

Pour les fonctionnalités métier, plusieurs bibliothèques sont utilisées :

• ZXing pour la génération et lecture des QR codes (payload : VMS|V{refVoucher}|R{refRequest}|{value}|{expiry}) ;

• PDFBox pour la génération des vouchers PDF ;

• Jakarta Mail pour l'envoi d'emails (notifications et vouchers) ;

• Apache POI pour l'export des rapports en Excel (.xlsx).

Enfin, PostgreSQL est utilisé comme base de données relationnelle avec des scripts SQL pour l'initialisation et les migrations.

Partie 3- Réalisation technique

3.1 Architecture globale

L'architecture adopte une structure en couches simples mais efficaces :

•Point d'entrée : Main.java lance MainApp via Application.launch()
•Couche présentation (UI) : MainApp.java gère l'écran de login, la navigation, les pages métier et les appels SQL
•Couche accès données : Database.java fournit une abstraction JDBC minimale (connexion, query paramétrée, update, insert avec retour d'ID)
•Couche modèle : POJOs dans models/ (Request, Voucher, Client, Branch, Company, User) servant de supports de transport entre SQL et UI
•Couche services utilitaires : QRUtil.java, VoucherPdfGenerator.java, EmailService.java, RefUtil.java.

Le flux global :
l'utilisateur interagit avec JavaFX (MainApp) → MainApp exécute des requêtes SQL via Database → les données sont chargées dans des objets modèles → lors des traitements avancés, QRUtil construit le payload, VoucherPdfGenerator génère le PDF, EmailService envoie le message → les statuts sont mis à jour en base.

Partie 3

3.2 Base de données

La base relationnelle structure les données métier et enforce les dépendances. Les cinq tables principales sont : users (rôle, statut), clients (email unique, statut actif/inactif), company et branch (organisation des points de redemption), requests (cœur du workflow, champs de suivi paiement/approbation/validation), et vouchers (statut actif/redeemed/expiré, informations de redemption).

Le schéma met en place foreign keys avec politiques ON UPDATE/ON DELETE, index sur colonnes de recherche, et données seed pour les tests. La présence de scripts de migration (migrate.sql) montre une évolution réelle du besoin. L'alignement schéma-domaine permet l'audit : une request peut produire plusieurs vouchers, chaque redemption est rattachée à un utilisateur et une branche, la chaîne de responsabilité est conservée.

3.3 Logique métier critique

Cinq mécanismes structurent la logique métier :

•Vérification paiement avant progression : markPaid() vérifie la présence du PDF, bloque si statut incompatible, met à jour transactionnellement et notifie les approvers
•Approbation contrôlée : approveRequest() exige un paiement validé et est limitée aux profils autorisés
•Génération idempotente : generateAndSendVouchers() contrôle l'existence de vouchers avant création, évitant les régénérations accidentelles

Partie 3

•Redemption avec validation QR : combinaison entrée manuelle et caméra avec thread dédié daemon, hints ZXing limités, debouncing temporel et fallback manuel

•Reporting actionnable : statistiques et tableaux opérationnels avec export Excel multi-feuilles.

3.4 Difficultés rencontrées et solutions

Respect du workflow	Conditions explicites dans markPaid, approveRequest, generateAndSendVouchers + contrôle role-based → réduction du risque de non-conformité
Double génération	Vérification préalable du nombre de vouchers associés à ref_request → cohérence financière garantie
Lecture QR temps réel	Thread daemon dédié + hints ZXing limités au QR + debounce + fallback manuel → expérience stable
PDF professionnel	Composition graphique PDFBox avec zones structurées (header, valeur, QR, validité, signature) → document exploitable en production
Email SMTP fiable	EmailService avec méthodes métier dédiées par profil et pièce jointe conditionnelle → communication automatisée

Partie 4 – Résultats obtenus

4.1 Présentation de l'application finale

Le produit final est une application desktop complète couvrant l'ensemble du cycle de vie des vouchers. Les modules livrés sont :

•Authentification et rôles : connexion utilisateur avec rôle et statut
•Dashboard (Superuser) : KPIs système (demandes, paiements, vouchers actifs/redeemed/expirants)
•Requests : création, consultation, upload preuve, mark paid, approbation, génération + envoi
•Vouchers : suivi des vouchers émis et de leur statut
•Redeem : scan QR caméra ou saisie manuelle, validation et confirmation
•Administration : gestion clients, users, branches, companies
•Reports : tableaux, statistiques, export Excel multi-feuilles.

Partie 4

4.2 Conformité aux besoins exprimés

Besoin exprimé	Réponse VMS	Niveau
Centraliser la gestion des vouchers	Base PostgreSQL centralisée + UI unique	**Atteint**
Structurer les validations	Workflow en étapes + contrôles préalables	**Atteint**
Sécuriser les vouchers	QR unique + statuts + redemption	**Atteint**
Générer des documents	PDF voucher automatisé	**Atteint**
Communication client	Envoi email automatisé avec pièce jointe	**Atteint**
Traçabilité de redemption	date / branche / user de redemption	**Atteint**
Pilotage et statistiques	Dashboard + Reports + export Excel	**Atteint**

Points forts

•Bonne couverture du cycle de vie complet

•Cohésion métier des modules

•Base solide pour extension Phase 2 (API, mobile).

Limites identifiées

•Architecture UI monolithique pouvant ralentir l'évolution future

•Absence de couche API séparée pour un usage multi-clients.

Partie 4

4.3 Évaluation qualitative

Pertinence fonctionnelle	Élevée — le système couvre le cycle métier central avec mécanismes de contrôle explicites
Qualité d'exécution technique	Satisfaisante — intégration cohérente de technologies hétérogènes (UI, SQL, QR, PDF, SMTP, Excel), produit stable
Maturité de sécurité	En progression — mécanismes de rôles existants, renforcement recommandé (secrets, mots de passe, audit log)
Maintenabilité	Acceptable à court terme, prioritaire à moyen terme — refactoring progressif de MainApp.java nécessaire

Conclusion

Le projet VMS répond de manière concrète aux besoins de gestion de vouchers exprimés par l'organisation cliente. La solution obtenue n'est pas un simple prototype visuel : elle implémente un processus métier complet, avec règles de validation, production documentaire, notifications et suivi analytique.

Bilan technique et personnel

Sur le plan technique, le projet a permis de consolider une approche full-stack desktop autour de Java, SQL et bibliothèques métier spécialisées. Le système délivre une valeur immédiate : réduction des opérations manuelles, meilleure traçabilité et limitation des erreurs.

Sur le plan personnel, ce projet a renforcé la capacité à traduire un besoin métier en architecture logicielle, la rigueur de conception de workflow, la gestion d'intégrations externes (QR, PDF, SMTP, Excel) et la priorisation selon la valeur utilisateur.

Améliorations futures

1. Externaliser la configuration sensible (variables d'environnement, suppression des secrets en dur) ;
2. Renforcer la sécurité (hash des mots de passe, authentification renforcée, journalisation d'audit) ;
3. Automatiser les tests (unitaires, intégration SQL, scénarios de non-régression) ;
4. Évoluer vers une architecture multi-canal (API backend, client web/mobile pour les fonctions terrain).

Annexes

Références techniques du dépôt

•README.md

•pom.xml

•schema.sql

•migrate.sql

•src/main/java/com/vms/Main.java

•src/main/java/com/vms/ui/MainApp.java

•src/main/java/com/vms/db/Database.java

•src/main/java/com/vms/util/QRUtil.java

•src/main/java/com/vms/util/VoucherPdfGenerator.java

•src/main/java/com/vms/util/EmailService.java